

Lab 7: `stackl` Code Generation

For this lab you will write a code generation visitor. All of the code generation must be performed in the visitor, not in the AST nodes. The same rules on casting, looping, and leaking implementation details from one class to another apply to this lab.

The `lab7.tar` file includes the following:

1. The script that runs tests has changed because it now needs to run your output through the `stackl` interpreter. Use the script `runc` instead of the older `runt`.
2. The `emit.*` files should be used by your code generation visitor to perform the actual code output.
3. `io320.sl` contains code needed to print values. The following function is defined, along with some helper functions:

```
int print(integer value);
```

To access the `stackl` assembler and interpreter, you need to modify your `PATH`. To do this, edit the file `.bashrc` in your home directory. Add the following two lines near the bottom:

```
PATH=$PATH:/home2/philip.howard/bin
export PATH
```

If you need help modifying your path, let me know. You may have to log out and back in again for the changes to be effective.

Your compiler will generate a `langout.sl` file. This file contains the `stackl` assembly program. The `runc` script will assemble that file using `slasm`. This produces a binary `langout.slb`. The `runc` script will then run that binary using the `stackl` interpreter.

If you want to reassemble your file, you can do so with the following command:

```
slasm -dbg -list langoutout.sl io320.sl
```

The `io320.sl` file contains the code that allows your program to print.

If you want to re-run your program, you can do so with the following command:

```
stackl langout
```

If you want to debug your code, you have two options:

```
stackl -opcodes langout
```

will print a trace listing every instruction that `stackl` executed.

The `stackl` environment includes a debugger with capability similar to `gdb`. Since your code was not compiled with the `stackl` C compiler, you do not have full symbolic debugging capability, but you can step through the assembly code (the code in your `langout.sl`) and examine registers and memory. To run the debugger, use the following command:

```
stackl -dbg langout
```

The debugger will complain that it cannot find your debug files. Answer “y” to the question and you will be in assembly debug mode. You can type “help” to get a list of commands. Some useful ones:

<code>ni</code>	Next Instruction. If you type this twice, you will be at the first instruction of your program.
<code>FP</code>	Print the value of the Frame Pointer
<code>SP</code>	Print the value of the Stack Pointer
<code>IP</code>	Print the value of the Instruction Pointer
<code>pi</code>	Print the current instruction
<code>print 0x123</code>	Print the contents of memory at hex location 0x123 (you must use hex notation)
<code>bi 123</code>	Break Instruction. Put a breakpoint at instruction at location 123. (See <code>langout.lst</code> to figure out what location your instructions were placed at.)
<code>c</code>	Continue to a breakpoint
<code><enter></code>	If you hit <code><enter></code> you will repeat the last command

Note: If you have “bad” code, you may cause the `stackl` machine to machine check (aka seg fault). If this happens, the debugger is supposed to take control (if you’re running with `-dbg`), but this functionality is new to the debugger and has not been fully tested.

Personally, I find running the interpreter with `-opcodes` generally easier than using the debugger, but your mileage may vary.